

解云月工作内容交接

kube-ovn

相关文档：

[操作手册](#)

[运维手册](#)

[性能测试](#)

[LB service使用指南](#)

[代码涉及组件](#)

[涉及资源](#)

[networkpolicy](#)

[service](#)

[默认route和policy route](#)

[ovn0](#)

[snat和dnat_and_snat](#)

[init](#)

[gc](#)

[未来计划](#)

[magnum](#)

kube-ovn [🔗](#)

相关文档：[🔗](#)

操作手册 [🔗](#)

[📖 kube-ovn 6.3.1 操作手册](#)

[📖 裸金属kube-ovn操作手册](#)

运维手册 [🔗](#)

[📖 kube-ovn运维手册](#)

性能测试 [🔗](#)

[📖 kube-ovn大规模测试](#)

LB service使用指南 [🔗](#)

[📖 LB Service使用手册](#)

代码涉及组件 [🔗](#)

cmd/cni -> 节点上的kube-ovn cni二进制

cmd/controller -> kube-ovn-controller

cmd/daemon -> kube-ovn-cni

cmd/webhook -> ovn-webhook

/run/openvswitch/kube-ovn-daemon.sock 节点上kube-ovn cni通过此socket和kube-ovn-cni交互

涉及资源 [🔗](#)

vpc对应 ovn的 logical router , ovn-nbctl lr-list

subnet对应 ovn的 logical switch, `ovn-nbctl ls-list`

ip对应ovn的 logical switch port, `ovn-nbctl lsp-list`

subnet和vpc连接后, 会分别创建logical_router_port(lrp)在router上和logical_switch_port(lsp)在switc上

查询lrp, `ovn-nbctl lrp-list`

查询具体的资源的详细信息 `ovn-nbctl find\list <resource type> <field-name>=<field-value>`

比如:

```
()[root@ovn-tool-0 /]# ovn-nbctl list logical_switch_port test1-77c9c965b8-dbprg.test
```

```
_uuid : 08842073-8b51-44e5-beed-4392572c7ccd
```

```
addresses : ["00:00:00:BC:D5:62 10.16.0.14"]
```

```
dhcpv4_options : []
```

```
dhcpv6_options : []
```

```
dynamic_addresses : []
```

```
enabled : [external_ids : {pod="test/test1-77c9c965b8-dbprg", vendor=kube-ovn}
```

```
ha_chassis_group : []
```

```
name : test1-77c9c965b8-dbprg.test
```

```
options : {}
```

```
parent_name : []
```

```
port_security : []
```

```
tag : []
```

```
tag_request : []
```

```
type : ""
```

```
up : true
```

或者

```
ovn-nbctl find logical_switch_port external_ids:vendor=kube-ovn
```

networkpolicy [🔗](#)

networkpolicy靠acl实现的

```
ovn-nbctl acl-list <ls-name/port group name>
```

port group中是networkpolicy过滤器匹配的pod集合, 即此acl会应用到哪些pod上

service [🔗](#)

service机制靠ovn lb实现的, ovn中 lb可以加在ls也可以加在lr上, kube-ovn中, lb直接应用到ls上, 所以kube-ovn的pod只能访问使用同样为kube-ovn的pod的service

```
()[root@ovn-tool-0 /]# ovn-nbctl ls-lb-list ovn-default
```

UUID LB PROTO VIP IPs

```
19af5dc5-2d28-496d-933b-4bf76fc5cc03 cluster-tcp-load tcp 10.233.18.253:80 10.16.0.16:80
```

正常会创建4个lb, 分别是tcp, udp, tcp-session和udp-session, 对应不同service类型

默认route和policy route [🔗](#)

vpc创建后，会默认添加route和policy-route

默认的route是为了pod能和k8s service和coredn service通信，pod请求svc地址时，转发到节点上的ovn0地址

```
()[root@ovn-tool-0 /]# ovn-nbctl lr-route-list 283a7ef1-e2ca-438d-975e-e1266c24c3b8
```

IPv4 Routes

10.233.0.1 100.64.0.5 dst-ip ecmp

10.233.0.1 100.64.0.6 dst-ip ecmp

10.233.0.1 100.64.0.7 dst-ip ecmp

10.233.0.3 100.64.0.5 dst-ip ecmp

10.233.0.3 100.64.0.6 dst-ip ecmp

10.233.0.3 100.64.0.7 dst-ip ecmp

0.0.0.0/0 172.18.0.1 dst-ip

默认的policy route是为了pod能和节点通信，是为了支持pod能访问hostnetwork的pod

```
()[root@ovn-tool-0 /]# ovn-nbctl lr-policy-list 283a7ef1-e2ca-438d-975e-e1266c24c3b8
```

Routing Policies

30000 ip4.dst == 10.51.51.27 reroute 100.64.0.4

30000 ip4.dst == 10.51.51.28 reroute 100.64.0.3

30000 ip4.dst == 10.51.51.29 reroute 100.64.0.2

30000 ip4.dst == 10.51.51.5 reroute 100.64.0.730000 ip4.dst == 10.51.51.61 reroute 100.64.0.8

30000 ip4.dst == 10.51.51.7 reroute 100.64.0.6

30000 ip4.dst == 10.51.51.8 reroute 100.64.0.5

ovn0 [🔗](#)

为了kubelet能和kube-ovn pod通信，所以节点上有一个ovn0设备，此ovn0设备的地址属于join网络，join网络会

加入到所有的vpc下

默认情况下会创建下面的vpc和subnet

```
[root@node-5 ~]# kubectl get vpc
```

NAME STANDBY SUBNETS NAMESPACES

```
ovn-cluster true ["ovn-default","join"]
```

```
[root@node-5 ~]# kubectl get subnet
```

NAME PROVIDER VPC PROTOCOL CIDR PRIVATE NAT DEFAULT GATEWAYTYPE

V4USED V4AVAILABLE V6USED V6AVAILABLE EXCLUDEIPS

```
join ovn ovn-cluster IPv4 100.64.0.0/16 false false false distributed 7 65526 0
```

```
0 ["100.64.0.1"]
```

```
ovn-default ovn ovn-cluster IPv4 10.16.0.0/16 false true true distributed 2 65531
```

```
0 0 ["10.16.0.1"]
```

snat和dnat_and_snat [🔗](#)

snat可以一对多，一个地址对应的多个pod，pod出集群时，同一为此地址

dnat_and_snat即fip，一对一关系

两个不能同时设置

一个公网对应一个fip资源，记录了proton port，pod name和fip地址的对应关系，其中forbidden ip也会记录非pod使用的地址

```
[root@node-5 ~]# kubectl get fips
```

```
NAME EXTERNALNETWORKNAME
```

```
01bb8814-8136-4e85-b9b2-3447c0bf3bf7 public_net
```

EXTERNAL_IP是fip地址，LOGICAL_IP是pod地址

```
()[root@ovn-tool-0 /]# ovn-nbctl lr-nat-list 283a7ef1-e2ca-438d-975e-e1266c24c3b8
```

```
TYPE EXTERNAL_IP EXTERNAL_PORT LOGICAL_IP EXTERNAL_MAC LOGICAL_PORT
```

```
dnat_and_snat 172.18.0.201 192.168.0.170
```

```
snat 172.18.0.144 192.168.0.0/24
```

```
snat 172.18.0.144 0.0.0.0/0
```

```
snat 172.18.0.21 10.16.0.14
```

init [🔗](#)

kube-ovn-controller启动时会先init资源，包括默认vpc ovn-cluster和默认subnet ovn-default以及节点子网join

我们产品里默认vpc可以事先指定一个已存在的proton或者kube-ovn会自动创建一个proton router，通过参数--

cluster-router控制，查到就直接使用，查不到自动创建

ipam在init逻辑中会从ip资源中回放pod和ip的对应关系到，维护一份对应关系到内存中，从而加快ip的分配

gc [🔗](#)

gc ls, lr, lsp, LB, staticRoute等等

fip中也会gc fip port和fip cr的pod和fip对应关系

未来计划 [🔗](#)

升级kube-ovn，跟上社区，但是跨度太大，私有patch太多。只能尽量升级，建议先把操作ovn从command切换

到ovndb-client，后续再按需升级feature和fix bug

magnum [🔗](#)

重点关注magnum/drivers/k8s_fedora_atomic_v1下的实现，产品都是基于此驱动实现的

主模板magnum/drivers/k8s_fedora_atomic_v1/templates/kubecuster.yaml，其它模板都是镶嵌在此模板中，其中定义了master节点，etcd节点，worker节点，etcd-discovery节点，以及网络，lb等IaaS资源

其中k8s的部署是等待节点启动后，通过cloud-init跑出来的，k8s周边组件的部署通过heat-container-agent这个service部署的

k8s部署通过type: OS::Heat::MultipartMime这种资源部署的

具体的脚本在magnum/drivers/common/templates路径下（忽略swarm，没有用到）

heat-container-agent服务部署的脚本是通过type: OS::Heat::SoftwareConfig资源定义的

常用命令

openstack coe clusetr list

openstack coe cluster show

heat stack-list

heat stack-list -asgn

heat resource-list -n 2